

Тестовое задание для Python/Django-разработчика

Общая информация

Необходимо исправить функцию формирования товарного YML-фида для Яндекс Директа.

В предоставленном ниже коде фид генерируется с ошибками:

- в него попадают товары, которые не должны выгружаться;
- некорректно передаются цены и наличие;
- категории дублируются;
- старая цена выводится без проверки;
- специальные символы могут повредить XML;
- отсутствует корректная обработка отдельных полей.

Полноценный Django-проект создавать не требуется. Основная часть задания выполняется в виде обычного Python-модуля.

Официальный формат YML предусматривает корневой элемент `<yml_catalog>`, дату генерации в формате `YYYY-MM-DD hh:mm`, размещение валют и категорий перед предложениями, а также уникальные идентификаторы товаров. Цена передаётся с точкой, а старая цена должна быть выше текущей.

Что необходимо сделать

1. Скопировать приведённый ниже код в файл `feed_task.py`.
2. Исправить функцию `build_yml()`.
3. Не изменять исходные данные `CATEGORIES` и `PRODUCTS`.
4. Не изменять сигнатуру функции:

```
def build_yml(products, categories, generated_at):
```

5. Добавить автоматические тесты.
6. Показать пример подключения исправленной функции к Django view.

Допускается добавлять вспомогательные функции и использовать стандартную библиотеку Python.

Исходный код

```
from datetime import datetime
```

```
CATEGORIES = [
```

```
{
```

```
    "id": 1,
```

```
    "name": "Чай",
```

```
    "is_active": True,
```

```
},
```

```
{
```

```
    "id": 2,
```

```
    "name": "Посуда",
```

```
    "is_active": True,
```

```
},
```

```
{
```

```
    "id": 3,
```

```
    "name": "Подарочные наборы",
```

```
    "is_active": False,
```

```
},
```

```
]
```

```
PRODUCTS = [
```

```
{
```

```
    "id": 101,
```

```
    "name": "Чай "Лес & травы" <сбор №1>",
```

```
    "slug": "les-i-travy",
```

```
"category_id": 1,

"price": "490.00",

"old_price": "590.00",

"stock": 12,

"description": "Вкус: мята & чабрец > классический чай",

"image_url": "https://example.test/media/tea-101.jpg",

"is_active": True,
},
{

"id": 102,

"name": "Чайник стеклянный",

"slug": "glass-teapot",

"category_id": 2,

"price": "1500.00",

"old_price": "1400.00",

"stock": 0,

"description": "Стеклянный чайник объёмом 800 мл",

"image_url": "https://example.test/media/teapot-102.jpg",

"is_active": True,
},
{

"id": 103,

"name": "Скрытый товар",

"slug": "hidden-product",

"category_id": 1,

"price": "350.00",
```

```
"old_price": None,  
"stock": 5,  
"description": "Товар отключён администратором",  
"image_url": "https://example.test/media/product-103.jpg",  
"is_active": False,  
},  
{  
  "id": 104,  
  "name": "Пробник чая",  
  "slug": "tea-sample",  
  "category_id": 1,  
  "price": "0.00",  
  "old_price": None,  
  "stock": 30,  
  "description": "Бесплатный пробник",  
  "image_url": "https://example.test/media/product-104.jpg",  
  "is_active": True,  
},  
{  
  "id": 105,  
  "name": "Чашка фарфоровая",  
  "slug": "porcelain-cup",  
  "category_id": 2,  
  "price": "700.00",  
  "old_price": "900.00",  
  "stock": 4,
```

```
"description": "Фарфоровая чашка",
"image_url": None,
"is_active": True,
},
{
  "id": 106,
  "name": "Подарочный набор",
  "slug": "gift-set",
  "category_id": 3,
  "price": "2500.00",
  "old_price": "3000.00",
  "stock": 2,
  "description": "Товар находится в неактивной категории",
  "image_url": "https://example.test/media/product-106.jpg",
  "is_active": True,
},
{
  "id": 107,
  "name": "Чай улун молочный",
  "slug": "milk-oolong",
  "category_id": 1,
  "price": "700.50",
  "old_price": None,
  "stock": 3,
  "description": "",
  "image_url": "https://example.test/media/product-107.jpg",
```

```
        "is_active": True,  
    },  
]
```

```
def build_yaml(products, categories, generated_at):  
    xml = '<?xml version="1.0" encoding="UTF-8"?>'  
  
    xml += f'<yaml_catalog date="{generated_at}">'  
    xml += "<shop>"  
  
    xml += "<name>Test Shop</name>"  
    xml += "<company>Test Company</company>"  
    xml += "<url>https://example.test</url>"  
  
    xml += '<currencies><currency id="RUB" rate="1"/></currencies>'  
  
    xml += "<categories>"  
  
    for product in products:  
        category = next(  
            category  
            for category in categories  
            if category["id"] == product["category_id"]  
        )
```

```
xml += (  
    f'<category id="{category["id"]}">'   
    f'{category["name"]}'   
    f'</category>'   
)
```

```
xml += "</categories>"
```

```
xml += "<offers>"
```

```
for product in products:
```

```
    xml += (  
        f'<offer id="{product["id"]}" '   
        f'available="{product["stock"]}">'   
    )
```

```
    xml += (  
        f'<url>'   
        f'https://example.test/products/{product["slug"]}'   
        f'</url>'   
    )
```

```
    price = product["price"].replace(".", ",")
```

```
    xml += f'<price>{price}</price>'
```

```
    if product["old_price"]:
```

```
xml += f'<oldprice>{product["old_price"]}</oldprice>'
```

```
xml += "<currencyId>RUB</currencyId>"
```

```
xml += f'<categoryId>{product["category_id"]}</categoryId>'
```

```
xml += f'<picture>{product["image_url"]}</picture>'
```

```
xml += f'<name>{product["name"]}</name>'
```

```
xml += f'<description>{product["description"]}</description>'
```

```
xml += "</offer>"
```

```
xml += "</offers>"
```

```
xml += "</shop>"
```

```
xml += "</yml_catalog>"
```

```
return xml
```

```
if __name__ == "__main__":
```

```
    result = build_yml(
```

```
        products=PRODUCTS,
```

```
        categories=CATEGORIES,
```

```
        generated_at=datetime(2026, 6, 18, 12, 0),
```

```
    )
```

```
print(result)
```

Требования к исправленной функции

1. Результат работы

Функция `build_yaml()` должна возвращать строку с корректным XML-документом в кодировке UTF-8.

XML должен успешно разбираться стандартным парсером:

```
import xml.etree.ElementTree as ET
```

```
ET.fromstring(result)
```

В результате не должно возникать исключений.

2. Дата генерации

Значение `generated_at` необходимо преобразовать в формат:

2026-06-18 12:00

Ожидаемый корневой элемент:

```
<yaml_catalog date="2026-06-18 12:00">
```

3. Правила включения товаров

В фид включаются только товары, которые одновременно соответствуют следующим условиям:

- товар активен;
- категория товара активна;
- название товара не пустое;
- цена товара больше нуля;
- указана ссылка на изображение;
- ссылка на изображение начинается с `http://` или `https://`.

Согласно предоставленным исходным данным, в фид должны попасть только товары:

101, 102 и 107

Товары 103, 104, 105 и 106 в фиде присутствовать не должны.

4. Категории

В блоке `<categories>` должны находиться только категории товаров, фактически попавших в фид.

Ожидаются категории:

1 — Чай

2 — Посуда

Каждая категория должна присутствовать только один раз.

Порядок категорий:

1, 2

5. Порядок товаров

Товары в блоке `<offers>` должны быть отсортированы по идентификатору.

Ожидаемый порядок:

101, 102, 107

6. Наличие товара

Атрибут `available` должен иметь только строковое значение `true` или `false`.

Правило:

- `true`, если `stock > 0`;
- `false`, если `stock == 0`.

Ожидаемый результат:

```
<offer id="101" available="true">
```

```
<offer id="102" available="false">
```

```
<offer id="107" available="true">
```

Яндекс использует значения `true` и `false` для передачи возможности купить товар.

7. Цена

Цена должна:

- быть больше нуля;
- выводиться с точкой в качестве десятичного разделителя;
- содержать два знака после точки.

Ожидаемые значения:

```
<price>490.00</price>
```

```
<price>1500.00</price>
```

```
<price>700.50</price>
```

8. Старая цена

Элемент `<oldprice>` необходимо добавлять только в том случае, если старая цена:

- указана;
- больше нуля;
- строго больше текущей цены.

Поэтому:

- у товара `101` элемент `<oldprice>` должен присутствовать;
- у товара `102` элемент `<oldprice>` должен отсутствовать, поскольку `1400.00` меньше текущей цены `1500.00`;
- у товара `107` элемент должен отсутствовать.

Ожидаемое значение для товара `101`:

```
<oldprice>590.00</oldprice>
```

Официальные требования также предусматривают, что старая цена должна быть выше текущей.

9. Формирование предложения

Каждое предложение должно содержать:

```
<offer id="101" available="true">
```

```
<url>https://example.test/products/les-i-travy/</url>
```

```
<price>490.00</price>
```

```
<oldprice>590.00</oldprice>
```

```
<currencyId>RUB</currencyId>
```

```
<categoryId>1</categoryId>
```

```
<picture>https://example.test/media/tea-101.jpg</picture>
```

```
<name>Чай "Лес & травы" &lt;сбор №1&gt;</name>
```

```
<description>Вкус: мята & травы</description>
```

```
</offer>
```

Пример приведён для понимания структуры. Способ форматирования, отступы и переносы строк могут отличаться.

10. Специальные символы

Данные необходимо добавлять в XML безопасным способом.

Название и описание товара 101 содержат:

&

<

>

"

Итоговый документ не должен повреждаться.

После разбора XML значение названия должно полностью совпадать с исходным:

```
name_element.text == 'Чай "Лес & травы" <сбор №1>'
```

Не рекомендуется формировать XML простой конкатенацией строк.

11. Описание

Если описание товара пустое, элемент `<description>` добавлять не нужно.

Поэтому у товара 107 элемент `<description>` должен отсутствовать.

12. Обработка ошибок

Не допускается скрывать ошибки конструкцией:

```
try:
```

```
...
```

except Exception:

pass

Некорректные товары необходимо отфильтровывать по понятным правилам.

Наличие одного некорректного товара не должно приводить к прекращению формирования всего фида.

13. Ограничения

Нельзя:

- изменять содержимое `PRODUCTS`;
- изменять содержимое `CATEGORIES`;
- прописывать итоговый XML вручную;
- удалять проблемные товары из исходного списка;
- возвращать заранее подготовленную статическую строку;
- использовать идентификаторы `101`, `102` и `107` как прямое условие отбора.

Можно:

- использовать `xml.etree.ElementTree`;
- использовать другие стандартные модули Python;
- добавлять вспомогательные функции;
- добавлять собственные проверки данных;
- изменять внутреннюю реализацию `build_yml()`.